Reinforcement Learning for LLM

Bilibili 以往的月

Homepage

2026 年 4 月 16 日

目录

1. Introduction
2. Reinforcement Learning: Basics
3. Reinforcement Learning: Bellman Equation
4. Reinforcement Learning: Policy Evaluation
5. Reinforcement Learning: Value based methods
6. Reinforcement Learning: Policy based methods
7. Reinforcement Learning: Actor-Critic methods
8. RL for LLM: TRPO and PPO
9. RL for LLM: DPO
10. RL for LLM: GRPO
11. References

目录

1. Introduction
2. Reinforcement Learning: Basics
3. Reinforcement Learning: Bellman Equation
4. Reinforcement Learning: Policy Evaluation
5. Reinforcement Learning: Value based methods
6. Reinforcement Learning: Policy based methods
7. Reinforcement Learning: Actor-Critic methods

The Goal of This Tutorial

The Goal of This Tutorial

（建模）LLM 如何与 RL 对应？（算法）GRPO, PPO, DPO 背后的原理是什么？（实验）如何实现？

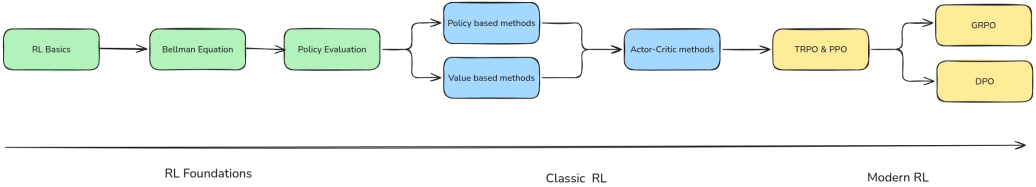

图: Pipeline of this RL-LLM tutorial series

References

- Reinforcement Learning for Large Language Models
- 强化学习的数学原理

目录

1 Introduction

2 Reinforcement Learning: Basics

3 Reinforcement Learning: Bellman Equation

4 Reinforcement Learning: Policy Evaluation

5 Reinforcement Learning: Value based methods

6 Reinforcement Learning: Policy based methods

7 Reinforcement Learning: Actor-Critic methods

What is Reinforcement Learning?

Core Idea

让 agent 通过与环境交互，学习到最优策略。

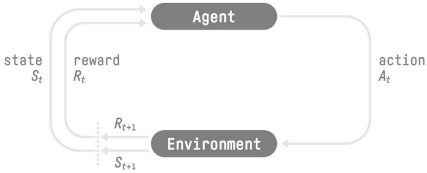

图: Illustration of the RL process

定义 (Reinforcement Learning)

通过构建可以与环境进行交互的 agent 来解决控制和决策任务，交互的方式为 agent 执行 action，然后环境给予反馈。

Markov Decision Process (MDP)

定义 (Markov Decision Process (MDP))

MDP 是描述强化学习的数学模型，形式化为：

- **状态 (State)**: $s_t \in \mathcal{S}$, $\mathcal{S}$ 为状态空间。
- **动作 (Action)**: $a_t \in \mathcal{A}, \mathcal{A}$ 为动作空间。
- **奖励 (Reward)**: $r_t \in \mathbb{R}$，环境对 agent 执行动作 a_t 的反馈。
- **终止时间 (Terminal Time)**: T，且定义 $s_T = \langle \text{term} \rangle$ 为终止状态。
- **初始状态分布 (Initial State)**: $s_0 \sim p_0$.
- **状态转移概率 (Transition Probability)**: $(r_t, s_{t+1}) \sim p(\cdot, \cdot \mid s_t, a_t)$.
- **马尔可夫性 (Markov Property)**:
  $p(s_{t+1} \mid s_t, a_t, \ldots) = p(s_{t+1} \mid s_t, a_t)$, $p(r_t \mid s_t, a_t, \ldots) = p(r_t \mid s_t, a_t)$.

Trajectory

定义 (Trajectory)

Trajectory (rollout) 定义为:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_{T-1}, a_{T-1}, r_{T-1}, s_T) \tag{1}$$

轨迹的概率分布写为如下形式:

$$p(\tau \mid s_0, \pi, p) = p_0(s_0) \prod_{i=1}^{T-1} \pi(a_t \mid s_t) p(r_t, s_{t+1} \mid r_t, s_t) \tag{2}$$

Remark

- 轨迹的概率分布是由初始状态分布，策略，以及环境模型共同决定的，我们可以将其记为 $\tau \sim (p_0, \pi, p)$.
- 我们在这个 tutorial 中仅考虑 finite horizon MDP, 即 $T < \infty$.

Expected Return

定义 (Expected Return)

Expected (discounted) return 定义为:

$$R(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t \tag{3}$$

其中 $\gamma \in (0, 1]$ 是 discount factor, 用于将未来的奖励折现到当前时刻。
我们还会使用另一个符号 G_0 来表示 expected return[a], 即

$$G_0 = \sum_{t=0}^{T-1} \gamma^t r_t \tag{4}$$

G_0 存在如下递推关系

$$G_0 = r_0 + \gamma G_1 \tag{5}$$

[a] 后续我们将会交错使用 G_0 和 $R(\tau)$ 来表示 expected return.

Objective of RL

Reward Hypothesis

所有的目标都可以被描述为最大化 expected return.

Objective

RL 的最终目标为最大化 expected return. 形式化为

$$\max_{\pi} \quad \mathbb{E}^{\pi}_{\tau\sim(p_0,\pi,p)} R(\tau) \tag{6}$$

Connection with LLM

LLM 的建模方式:

$$y_i \sim \mathrm{LLM}(\cdot \mid y_{<i}, x) \tag{7}$$

其中 x 是输入的 prompt, $y_{<i}$ 是已经生成的部分，y_i 是下一个要生成的 token.

RL	LLM
policy π	LLM $\mathrm{LLM}(\cdot \mid y_{<i}, x)$
initial state s_0	prompt x
action a_t	current token y_i
state s_t	context $(y_{<i}, x)$

Remark

注意到我们并没有对 environment 和 reward 进行建模，我们可以这样进行理解:

1. LLM 中的 environment 仅仅是对 token 进行简单拼接, $s_{t+1} = \mathrm{concate}(y_i, s_t)$.
2. reward 由 verifier, reward model (PRM, ORM) 或者 human feedback 给定

Action Space and State Space for LLM

action = token index $y_i \Rightarrow$ Action space = vocabulary of the LLM

state = context $(y_{<i}, x) \Rightarrow$ State space = all possible context of the LLM

表: 主流大语言模型词表大小 $|V|$ 对比

模型类别	模型名称	词表大小
商业闭源	GPT-5	∼200,000
	Claude Sonnet 3.7	∼100,000
	Gemini 3.0	∼256,144
主流开源模型	Qwen 3	151,936
	DeepSeek-V3	129,280
	MiniMax-M2.5	200,064
	GLM-5	154,880

LLM 中，action space ($|V|$) 非常大，state space ($|V|^i$) 更大。

Takeaway

- RL 的基础是 MDP, 其核心假设是 markov property, 即未来的状态和奖励只与当前状态和动作有关，与历史状态和动作无关。
- RL 通过 reward hypothesis, 将目标转化为最大化 expected return.
- 从 RL 角度来看，LLM 本身是一个 policy, 其初始状态为 prompt, 动作空间为 vocabulary 中的 token index, 奖励由多方面给定，environment 仅仅是对 token 进行简单拼接。
- 虽然 action space 和 state space 是离散的，但是其数量级非常大，因此我们通常使用函数逼近来近似 value function 和 Q function.

目录

1 Introduction

2 Reinforcement Learning: Basics

3 Reinforcement Learning: Bellman Equation

4 Reinforcement Learning: Policy Evaluation

5 Reinforcement Learning: Value based methods

6 Reinforcement Learning: Policy based methods

7 Reinforcement Learning: Actor-Critic methods

Why Bellman Equation?

Motivation

在强化学习中，我们关心的不是某一步立即得到的 reward，而是 agent 从当前状态出发，在未来持续决策后能够获得的累计回报。

Difficulty

累计回报依赖于整条未来轨迹：后续会到达什么状态、采取什么动作、获得什么奖励，都是随机的。因此直接分析 expected return 往往比较困难。

Core Idea

将长期回报写成“当前一步的收益 + 下一状态的未来价值”，把一个全局问题转化为递推问题。

$$\text{long-term value} = \text{immediate reward} + \text{discounted future value} \tag{8}$$

Value function and Q function

定义 (Value function)

Value function **定义为**:

$$V^{\pi}(s) = \mathbb{E}^{\pi}[G_0 \mid s_0 = s] \tag{9}$$

value function 的具体含义为: **agent 从当前状态** s **出发，一直遵循当前策略** π**，最后获取到的 expected return**.

定义 (Q function)

Q function **定义为**:

$$Q^{\pi}(s, a) = \mathbb{E}^{\pi}[G_0 \mid s_0 = s, a_0 = a] \tag{10}$$

Q function 的具体含义为: **agent 从当前状态** s **出发，先执行动作** a**，然后一直遵循当前策略** π**，最后获取到的 expected return**.

[0]在不影响理解的情况下，我们省略 $\tau \sim (p_0, \pi, p)$ 的标记

Relationship between value function and Q function

Relationship between value function and Q function

Value function 和 Q function 之间存在如下关系：

$$V^{\pi}(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}[Q^{\pi}(s,a)] \tag{11}$$

即 value function 是 Q function 的期望。

证明.

$$\begin{aligned} V^{\pi}(s) &= \mathbb{E}[G_0 \mid s_0 = s] && (12)\\ &= \sum_{a\in\mathcal{A}} \mathbb{E}\left[G_0 \mid s_t = s, a_0 = a\right] \pi(a \mid s_0 = s) && (13)\\ &= \sum_{a\in\mathcal{A}} Q^{\pi}(s,a)\pi(a \mid s) = \mathbb{E}_{a\sim\pi(\cdot|s)}[Q^{\pi}(s,a)]. && (14) \end{aligned}$$

Iterative property

Iterative property

Value function 和 Q function 存在如下递推关系

$$V^{\pi}(s) = \mathbb{E}_{a\sim\pi(\cdot|s),\ (r,s')\sim p(\cdot,\cdot|s,a)}[r + \gamma V^{\pi}(s') \mid s] \tag{15}$$

$$Q^{\pi}(s,a) = \mathbb{E}_{(r,s')\sim p(\cdot,\cdot|s,a),\ a'\sim\pi(\cdot|s')}[r + \gamma Q^{\pi}(s',a') \mid s,a] \tag{16}$$

证明.

$$V^{\pi}(s) = \mathbb{E}^{\pi}[r_0 + \gamma G_1 \mid s_0 = s] \tag{17}$$

$$= \mathbb{E}_{a\sim\pi(\cdot|s),\ (r,s')\sim p(\cdot,\cdot|s,a)}\left[\mathbb{E}^{\pi}[r + \gamma G_1 \mid s,a,r,s'] \mid s\right] \tag{18}$$

$$\overset{\text{Markov}}{=} \mathbb{E}_{a\sim\pi(\cdot|s),\ (r,s')\sim p(\cdot,\cdot|s,a)}\left[r + \gamma\mathbb{E}^{\pi}[G_1 \mid s',a] \mid s\right] \tag{19}$$

$$= \mathbb{E}_{a\sim\pi(\cdot|s),\ (r,s')\sim p(\cdot,\cdot|s,a)}\left[r + \gamma V^{\pi}[s') \mid s\right] \tag{20}$$

□

Bellman Equation (Value function)

定理 (Bellman Equation)

令 π 为一个策略，假设 $\gamma \in (0,1)$, $|\mathcal{S}| < \infty$ 以及 $|r| \leq R < \infty, \mathrm{a.s.}$[a]. 那么 π 对应的 *value function* $V^\pi : \mathcal{S}^+ \to \mathbb{R}$ 存在，且满足 *Bellman equation:*

$$V^\pi(s) = \mathbb{E}^\pi_{a\sim\pi(\cdot|s),\ (r,s')\sim p(\cdot,\cdot|s,a)} \left[r_0 + \gamma V^\pi(s') \mid s_0 = s \right] \tag{21}$$

反之，如果存在函数 $V : \mathcal{S}^+ \to \mathbb{R}$ 满足 *Bellman equation*, 则 $V = V^\pi$.

[a] $\mathrm{a.s.}$ 表示 almost surely

Fixed Point Theorem*

定义 (Fix Point)

对于函数 $f\colon \mathbb{R}^n \to \mathbb{R}^n$, 如果一个点 $x^* \in \mathbb{R}^n$ 满足

$$f(x^*) = x^* \tag{22}$$

则我们称 x^* 是函数 f 的 **fixed point**.

定义 (Contraction mapping)

对于函数 $f\colon \mathbb{R}^n \to \mathbb{R}^n$, 如果存在 $\gamma \in (0,1)$ 满足

$$\|f(x_1) - f(x_2)\| \leq \gamma \|x_1 - x_2\|, \forall\ x_1, x_2 \in \mathbb{R}^n \tag{23}$$

则我们称 f 是一个 **contraction mapping**. 这里 $\|\cdot\|$ 是一个 matrix norm.

Fixed Point Theorem*

定理 (Fixed Point Theorem)

给定函数 $f : \mathbb{R}^n \to \mathbb{R}^n$, 如果 f 是一个 *contraction mapping*, 则 f 具有如下性质

1. *Existence:* 存在 *fixed point* $x^* \in \mathbb{R}^n$ 满足 $f(x^*) = x^*$.
2. *Uniqueness:* *fixed point* x^* 唯一。
3. *Algorithm:* 对任意 $x_0 \in \mathbb{R}^n$, 使用迭代算法 $x_{k+1} = f(x_k)$ 产生的序列 $\{x_k\}_{k=1}^{\infty}$ 收敛到 *fixed point* $x*$, 且收敛速度为指数级。

证明.

证明见附录 Frame 58 □

Bellman Equation Theorem–Proof

证明.

由 V^π 定义，我们有

$$\left|\sum_{t=0}^{\infty}\gamma^t r_t\right| \leq \sum_{t=0}^{\infty}\gamma^t R = \frac{R}{1-\gamma} < \infty \tag{24}$$

因此，$\sum_{t=0}^{\infty}\gamma^t r_t$ 是一个绝对收敛的序列，从而其期望存在且有界。
对于 $V^\pi(s)$，我们有

$$\begin{aligned}
V^\pi(s) &= \mathbb{E}^\pi\left[r_0 + \gamma G_1 \mid s_0 = s\right] && (25)\\
&= \mathbb{E}_{a\sim\pi(\cdot|s),(r_0,s')\sim p(\cdot,\cdot|s,a)}\left[\mathbb{E}^\pi\left[r_0 + \gamma G_1 \mid s, a, r_0, s'\right]\right] && (26)\\
&= \mathbb{E}_{a\sim\pi(\cdot|s),(r_0,s')\sim p(\cdot,\cdot|s,a)}\left[r_0 + \gamma\mathbb{E}^\pi\left[G_1 \mid s, a, r_0, s'\right]\right] && (27)\\
&= \mathbb{E}_{a\sim\pi(\cdot|s),(r_0,s')\sim p(\cdot,\cdot|s,a)}\left[r_0 + \gamma\mathbb{E}^\pi\left[G_1 \mid s'\right]\right] && (28)\\
&= \mathbb{E}_{a\sim\pi(\cdot|s'),(r_0,s')\sim p(\cdot,\cdot|s,a)}\left[r_0 + V^\pi(s')\right] && (29)
\end{aligned}$$

Bellman Equation Theorem–Proof

证明.

最后，我们证明方程解的唯一性，假设存在函数 $V: \mathcal{S}^+ \to \mathbb{R}$ 满足 Bellman equation. 我们定义 Bellman 算子 $\mathcal{T}^\pi$ 为

$$(\mathcal{T}^\pi V)(s) := \mathbb{E}_{a\sim\pi(\cdot|s),\ (r,s')\sim p(\cdot,\cdot|s,a_0)}\left[r + \gamma V(s') \mid s\right] \tag{30}$$

我们证明该算子是一个 contraction mapping, 考虑范数 $\|V\|_\infty = \max_{s\in\mathcal{S}} |V(s)|$, 我们有

$$\begin{aligned}
|(\mathcal{T}^\pi V_1)(s) - (\mathcal{T}^\pi V_2)(s)| &= \left|\mathbb{E}_{a\sim\pi(\cdot|s'),(r,s')\sim p(\cdot,\cdot|s,a_0)}\left[\gamma V_1^\pi(s') - \gamma V_2^\pi(s')\right]\right| && (31)\\
&\le \gamma \mathbb{E}_{a\sim\pi(\cdot|s'),(r,s')\sim p(\cdot,\cdot|s,a_0)} \left|V_1^\pi(s') - V_2^\pi(s')\right| && (32)\\
&\le \gamma \max_{s''\in\mathcal{S}} |V_1(s'') - V_2(s'')| = \gamma \|V_1 - V_2\|_\infty && (33)
\end{aligned}$$

上式对于任意 $s \in \mathcal{S}$ 都成立，因此

$$\|\mathcal{T}^\pi V_1 - \mathcal{T}^\pi V_2\|_\infty \le \gamma \|V_1 - V_2\|_\infty \tag{34}$$

由于 $\gamma < 1$, 从而 $\mathcal{T}^\pi$ 是一个 contraction mapping.
根据不动点定理，已知 V^π 是一个不动点，而不动点唯一，因此我们有 $V = V^\pi$. □

Bellman Equation (Q function)

定理 (Bellman Equation)

令 π 为一个策略，假设 $\gamma \in (0,1)$, $|\mathcal{S}| < \infty$ $|\mathcal{A}| < \infty$ 以及 $|r| \leq R < \infty, \text{a.s.}$. 那么 π 对应的 *Q-function* $Q^\pi : \mathcal{S}^+ \times \mathcal{A} \to \mathbb{R}$ 存在，且满足 *Bellman equation:*

$$Q^\pi(s,a) = \mathbb{E}_{(r,s')\sim p(\cdot,\cdot|s,a),\ a'\sim\pi(\cdot|s')}\left[r + \gamma Q(s',a') \mid s,a\right] \tag{35}$$

反之，如果存在函数 $Q : \mathcal{S}^+ \times \mathcal{A} \to \mathbb{R}$ 满足 *Bellman equation*, 则 $Q = Q^\pi$.

证明.

证明与 value function 的证明基本类似，我们这里略过。 □

Optimal Policy

定义 (Optimal Policy)

如果策略 π^* 满足

$$V^{\pi^*}(s) \geq V^{\pi}(s), \forall s \in \mathcal{S}^+, \forall \pi \tag{36}$$

则我们称策略 π^* 是 **optimal policy**. 对应的 V^{π^*} 和 Q^{π^*} 分别称之为 **optimal value function** 以及 **optimal Q-function**, 我们简记为 $V^* = V^{\pi^*}$, $Q^* = Q^{\pi^*}$.

Remark

注意 optimal policy 与状态 s 无关，并且 optimal policy 是不唯一的，但是所有的 optimal policy 对应的 optimal value function 是同一个。

Optimal Value Function and Optimal Q-Function

Relationship between V^* and Q^*

V^* 与 Q^* 之间存在如下关系

$$V^*(s) = \max_{a \in \mathcal{A}} Q^*(s, a) \tag{37}$$

证明.

我们先证明左边小于右边，再证明右边小于左边。首先，

$$V^*(s) = \mathbb{E}_{a_0 \sim \pi^*(\cdot|s_0)}[Q^*(s, a)] \leq \mathbb{E}_{a_0 \sim \pi^*(\cdot|s_0)}[\max_{a \sim \pi^*} Q^*(s, a)] = \max_{a \sim \pi^*} Q^*(s, a) \tag{38}$$

其次，令 $a^* \in \arg\max_a Q^*(s, a)$，令策略 π' 为确定性策略 $\pi'(a^* \mid s) = 1$，则

$$\max_a Q^*(s, a^*) = Q^{\pi'}(s, a^*) = V^{\pi'}(s) \leq V^*(s) \tag{39}$$

这样，我们就证明了上面的等式。 □

Bellman Optimality Equation Theorem

定理 (Bellman Optimality Equation)

假设 $\gamma \in (0,1)$, $|\mathcal{S}| < \infty$ $|\mathcal{A}| < \infty$ 以及 $|r| \leq R < \infty, \text{a.s.}$. 那么 *optimal value function* $V^* : \mathcal{S}^+ \to \mathbb{R}$ 存在，且满足 *Bellman optimality equation:*

$$V^*(s) = \max_{a \in \mathcal{A}} \mathbb{E}_{(r,s') \sim p(\cdot,\cdot|s,a)} \left[r + \gamma V^*(s') \mid s, a \right] \tag{40}$$

反之，如果存在函数 $V : \mathcal{S}^+ \to \mathbb{R}$ 满足 *Bellman optimality equation*, 则 $V = V^*$. 最后，我们可以给出一个 *optimal deterministic policy:*

$$\pi^*(s) = \arg\max_{a \in \mathcal{A}} \mathbb{E}_{(r,s') \sim p(\cdot,\cdot|s,a)} \left[r + \gamma V^*(s') \mid s, a \right] = \arg\max_{a \in \mathcal{A}} Q^*(s, a) \tag{41}$$

Bellman Optimality Equation Theorem–Proof

引理

令 π 为一个策略，$\mathcal{T}^{\pi}$ 和 $\mathcal{T}^{*}$ 分别是 *Bellman operator* 和 *Bellman optimality operator*. 对任意 $V : \mathcal{S}^{+} \to \mathbb{R}$, 我们有

$$\mathcal{T}^{\pi}(V) \leq \mathcal{T}^{*}(V) \tag{42}$$

进一步，对任意 $U : \mathcal{S}^{+} \to \mathbb{R}$, $V : \mathcal{S}^{+} \to \mathbb{R}$, 如果 $U \leq V$, 则

$$\mathcal{T}^{*}(U) \leq \mathcal{T}^{*}(V) \tag{43}$$

证明.

$$\begin{aligned}\mathcal{T}^{\pi}(V)(s) &= \mathbb{E}_{a\sim\pi(\cdot|s),\ (r,s')\sim p(\cdot,\cdot|s,a)}\left[r+\gamma V(s') \mid s\right] \leq \mathbb{E}_{a\sim\pi(\cdot|s)}\left[\max_{a'}\mathbb{E}_{(r,s')\sim p(\cdot,\cdot|s,a')}\left[r+\gamma V(s') \mid s\right]\right] \\ &= \max_{a'}\mathbb{E}_{(r,s')\sim p(\cdot,\cdot|s,a')}\left[r+\gamma V(s') \mid s\right] = \mathcal{T}^{*}(V)\end{aligned}$$

其次，令 $a^{*} \in \arg\max_{a}\mathbb{E}[r+\gamma U(s')]$, 那么

$$\begin{aligned}\mathcal{T}^{*}(U)(s) &= \mathbb{E}_{(r,s')\sim p(\cdot,\cdot|s,a')}\left[r+\gamma U(s') \mid s\right] \leq \mathbb{E}_{(r,s')\sim p(\cdot,\cdot|s,a'))}\left[r+\gamma V(s') \mid s\right] \\ &\leq \max_{a}\mathbb{E}_{(r,s')\sim p(\cdot,\cdot|s,a')}\left[r+\gamma V(s') \mid s\right] = \mathcal{T}^{*}(V)\end{aligned}$$

证毕.

Bellman Optimality Equation Theorem–Proof

证明.

我们首先证明存在性，我们定义

$$(\mathcal{T}^* V)(s) := \max_{a \in \mathcal{A}} \mathbb{E}_{(r,s') \sim p(\cdot,\cdot|s,a_0))} \left[r + \gamma V(s') \mid s, a \right] \tag{44}$$

我们证明 $\mathcal{T}^* V$ 是一个 contraction mapping.

$$\begin{aligned}
\left|(\mathcal{T}^* V_1)(s) - (\mathcal{T}^* V_2)(s)\right| &= \left| \max_{a \in \mathcal{A}} \mathbb{E}_{(r,s') \sim p(\cdot,\cdot|s,a_0))} \left[r + \gamma V_1(s') \mid s, a \right] - \max_{a \in \mathcal{A}} \mathbb{E}_{(r,s') \sim p(\cdot,\cdot|s,a_0))} \left[r + \gamma V_2(s') \mid s, a \right] \right| && (45) \\
&\leq \max_{a \in \mathcal{A}} \left| \mathbb{E}_{(r,s') \sim p(\cdot,\cdot|s,a)} \left[r + \gamma V_1(s') \mid s, a \right] - \mathbb{E}_{(r,s') \sim p(\cdot,\cdot|s,a)} \left[r + \gamma V_2(s') \mid s, a \right] \right| && (46) \\
&= \gamma \max_{a \in \mathcal{A}} \left| \mathbb{E}_{(r,s') \sim p(\cdot,\cdot|s,a)} \left[V_1(s') - V_2(s') \mid s, a \right] \right| && (47) \\
&\leq \gamma \max_{a \in \mathcal{A}} \mathbb{E}_{(r,s') \sim p(\cdot,\cdot|s,a)} \left[\left| V_1(s') - V_2(s') \right| \mid s, a \right] && (48) \\
&\leq \gamma \max_{a \in \mathcal{A}} \max_{s'' \in \mathcal{S}} \left| V_1(s'') - V_2(s'') \right| = \gamma \left\| V_1 - V_2 \right\|_\infty && (49)
\end{aligned}$$

因此，我们有

$$\left\| \mathcal{T}^* V_1 - \mathcal{T}^* V_2 \right\|_\infty \leq \gamma \left\| V_1 - V_2 \right\|_\infty \tag{50}$$

由于 $\gamma < 1$, 从而 $\mathcal{T}^*$ 是一个 contraction mapping. □

[0] 这里第一个不等式我们使用了结论 $|\max_s v(s) - \max_s u(s)| \leq \max_s |u(s) - v(s)|, \forall s \in \mathcal{S}$.

Bellman Optimality Equation Theorem–Proof

证明.

根据不动点定理，$\mathcal{T}^*$ 存在一个不动点，我们将其记为 V^*. 接下来，我们定义 π^* 为

$$\pi^*(s) = \arg\max_{a \in \mathcal{A}} \mathbb{E}_{(r,s') \sim p(\cdot,\cdot|s,a)} \left[r + \gamma V^*(s') \mid s, a \right] \tag{51}$$

此时，我们有

$$\begin{aligned}
V^*(s) &= \mathbb{E}^{\pi^*} \left[r_0 + \gamma V^*(s_1) \mid s \right] && (52)\\
&= \mathbb{E}^{\pi^*} \left[r_0 + \gamma \mathbb{E}^{\pi^*} \left[r_1 + \gamma V^*(s_2) \mid s_1 \right] \mid s \right] && (53)\\
&= \mathbb{E}^{\pi^*} \left[\mathbb{E}^{\pi^*} \left[r_0 + \gamma (r_1 + \gamma V^*(s_2)) \mid s_1 \right] \mid s \right] && (54)\\
&= \mathbb{E}^{\pi^*} \left[r_0 + \gamma r_1 + \gamma^2 V^*(s_2) \mid s \right] && (55)\\
&= \mathbb{E}^{\pi^*} \left[r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots \mid s \right] = V^{\pi^*}(s) && (56)
\end{aligned}$$

即 $V^* = V^{\pi^*}$. 现在我们证明 π^* 是最优策略，令 π 为任意一个策略，我们有

$$V^\pi \overset{\text{Lemma 14}}{\leq} V^* \overset{\text{BOE}}{=} \mathcal{T}^*(V^\pi) \overset{V^\pi \leq \mathcal{T}^*(V^\pi)}{\leq} (\mathcal{T}^*)^2(V^\pi) \leq \cdots \leq (\mathcal{T}^*)^k(V^\pi) \xrightarrow[k \to \infty]{\text{fixed-point / contraction}} V^* = V^{\pi^*}. \tag{57}$$

□

Bellman Optimality Equation Theorem (Q-function version)

定理 (Bellman Optimality Equation (Q-function version))

假设 $\gamma \in (0,1)$, $|\mathcal{S}| < \infty$ $|\mathcal{A}| < \infty$ 以及 $|r| \leq R < \infty, \text{a.s.}$. 那么 *optimal Q-function* $Q^* : \mathcal{S}^+ \times \mathcal{A} \to \mathbb{R}$ 存在，且满足 ***Bellman optimality equation**:*

$$Q^*(s,a) = \mathbb{E}_{(r,s')\sim p(\cdot,\cdot|s,a)}\left[r + \gamma \max_{a' \in \mathcal{A}} Q^*(s',a') \mid s,a\right] \tag{58}$$

反之，如果存在函数 $Q : \mathcal{S}^+ \times \mathcal{A} \to \mathbb{R}$ 满足 *Bellman optimality equation*, 则 $Q = Q^*$.

证明.

证明与 Bellman Optimality Equation Theorem 类似，这里不再赘述。 □

Value Iteration Algorithm

根据 BOE 定理的证明，我们知道

$$(\mathcal{T}^* V)(s) := \max_{a \in \mathcal{A}} \mathbb{E}_{(r,s') \sim p(\cdot,\cdot|s,a)} \left[r + \gamma V(s') \mid s, a \right] \tag{59}$$

是一个 contraction mapping, 因此我们可以构造函数序列

$$V, \mathcal{T}^* V, (\mathcal{T}^*)^2 V, \cdots, \tag{60}$$

由不动点定理，我们知道这个序列收敛到 V^*. 这样，我们就得到了 value iteration algorithm:

$$V^{k+1} = \mathcal{T}^* V^k \tag{61}$$

$$V^{k+1}(s) = \max_{a \in \mathcal{A}} \mathbb{E}_{(r,s') \sim p(\cdot,\cdot|s,a)} \left[r + \gamma V^k(s') \mid s, a \right] \tag{62}$$

$$\pi^{k+1}(s) = \arg\max_{a \in \mathcal{A}} \mathbb{E}_{(r,s') \sim p(\cdot,\cdot|s,a)} \left[r + \gamma V^k(s') \mid s, a \right] \tag{63}$$

Summary

1. 我们介绍了 value function 和 Q-function 的定义，以及它们之间的联系。这是强化学习的核心概念。
2. 我们介绍了 Bellman equation 和 Bellman optimality equation 的定义，BOE 定理是后续各种算法的核心。
3. 我们基于 BOE 定理，给出了 value iteration algorithm, 这是强化学习的基本算法。后续我们会对这个算法进行改进。

目录

1 Introduction

2 Reinforcement Learning: Basics

3 Reinforcement Learning: Bellman Equation

4 Reinforcement Learning: Policy Evaluation

5 Reinforcement Learning: Value based methods

6 Reinforcement Learning: Policy based methods

7 Reinforcement Learning: Actor-Critic methods

Why Policy Evaluation?

Motivation

在强化学习中，我们通常希望回答一个基本问题：

给定一个策略 π，它到底“好不好”？

更具体地说，我们希望知道 agent 在不同状态下遵循该策略时，未来能够获得多大的长期回报。

Difficulty

这个问题本质上对应于求解策略 π 的 value function V^π。虽然上一节已经知道 V^π 满足 Bellman equation，但当状态空间较大、动作空间较大，或者环境模型未知时，直接进行精确求解往往计算代价较高。

Core Idea

利用 Bellman equation、采样和函数逼近，高效地估计或拟合给定策略对应的 value function V^π。

Monte Carlo Introduction

Basic Idea

Monte Carlo (MC) 方法通过随机采样来估计期望值：

用样本平均来逼近期望

统计学保证了样本平均是期望的**无偏估计**。

In RL

对于 value function

$$V^\pi(s) = \mathbb{E}^\pi[R(\tau) \mid s_0 = s],$$

我们可以从 $s_0 = s$ 出发，按照策略 π 采样多条完整轨迹，并用这些轨迹 return 的平均值来近似 $V^\pi(s)$。

Remark

Monte Carlo 方法的特点是：简单、直观、无需环境模型，但必须等轨迹结束后才能得到一次完整的估计。

Monte Carlo

注意到

$$V^{\pi}(s) = \mathbb{E}^{\pi}\left[R(\tau) \mid s_0 = s\right] \tag{64}$$

我们可以利用 Monte Carlo (MC) 方法来进行估计，即从 $s_0 = s$ 出发，独立随机采样 N 条轨迹

$$\{s_0^{(i)}, a_0^{(i)}, r_0^{(i)}, \ldots, a_{T-1}^{(i)}, r_{T-1}^{(i)}, s_T^{(i)}\}, i = 1, \ldots, N \tag{65}$$

然后我们使用样本平均来近似期望

$$V^{\pi}(s) = \mathbb{E}^{\pi}\left[R(\tau) \mid s_0 = s\right] \approx \frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)}) \tag{66}$$

这样，我们就得到了一个 Monte Carlo estimator 来估计 $V^{\pi}(s)$. 如果 $\mathcal{S}$ 比较小，我们可以遍历 $s \in \mathcal{S}$ 来估计 V^{π}. 但是，如果 $\mathcal{S}$ 比较大，这种方法的计算复杂度较高。

Objective of Value based methods

为了解决状态空间较大或者连续时，直接使用 MC 方法计算复杂度较高的问题，我们可以使用函数逼近的方法来近似 V^π.

具体做法就是，我们需要使用神经网络 V_ϕ 来近似 V^π,

$$\min_\phi \mathcal{L}(\phi) = \mathbb{E}\left[\frac{1}{2}\left(V_\phi(s) - V^\pi(s)\right)^2\right] \tag{67}$$

然后我们可以使用随机梯度下降法来更新参数 ϕ.

[0]这里关于 s 的概率分布需要进一步讨论，请参考 < 强化学习的数学原理 >

Gradient Calculation

对上面的目标函数 (67)求导得到

$$
\begin{aligned}
\nabla_\phi \mathcal{L}(\phi) &= \mathbb{E}\left[\left(V_\phi(s) - V^\pi(s)\right) \nabla_\phi V_\phi(s)\right] && (68)\\
&= \mathbb{E}\left[\left(V_\phi(s) - \mathbb{E}^\pi\left[R(\tau) \mid s_0 = s\right]\right) \nabla_\phi V_\phi(s)\right] && (69)\\
&= \mathbb{E}\left[\mathbb{E}^\pi\left[\left(V_\phi(s) - R(\tau)\right) \nabla_\phi V_\phi(s) \mid s_0 = s\right]\right] && (70)\\
&= \mathbb{E}^\pi\left[\left(V_\phi(s) - R(\tau)\right) \nabla_\phi V_\phi(s) \mid s_0 = s\right] && (71)
\end{aligned}
$$

我们现在可以使用 MC 方法来估计梯度，即

$$
\nabla_\phi \mathcal{L}(\phi) \approx g := \frac{1}{N} \sum_{i=1}^{N} \left(V_\phi(s^{(i)}) - R(\tau^{(i)})\right) \nabla_\phi V_\phi(s^{(i)}) \tag{72}
$$

MC and SGD

Algorithm 1: Value function approximation with MC

```
while not converged do
    s_0 ~ p_0, t = 0;
    while s_t ≠ terminal do
        a_t ~ π(· | s_t);
        (r_t, s_{t+1}) ~ p(·, · | s_t, a_t);
        t ← t + 1;
    T = t;
    g = (V_φ(s_0) − R(τ)) ∇_φ V_φ(s_0);
    update φ using g with an optimizer;
```

Temporal Difference Learning–Intuition

为了理解 TD learning, 我们考虑下面这个例子

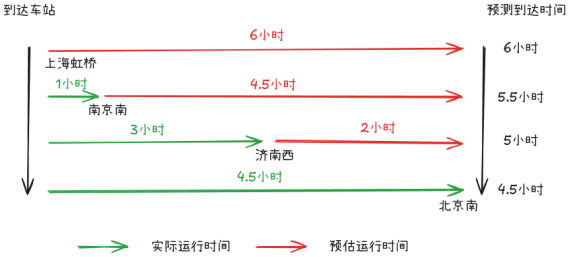

图: Illustration of TD learning

也就是说，TD learning 本质上是

通过新的信息和新的预测来更新旧的预测。

Temporal Difference Learning

Temporal difference (TD) learning 通过相邻两步的转换关系来近似 value function。注意到，

$$V^\pi(s) = \mathbb{E}^\pi[r_0 + \gamma V^\pi(s_1) \mid s_0 = s] \tag{73}$$

我们将

$$\overline{V}_t = r_t + \gamma V^\pi(s_{t+1}) \tag{74}$$

称为 TD target. 将

$$\delta_t = V^\pi(s_t) - (r_t + \gamma V^\pi(s_{t+1})) \tag{75}$$

称为 TD error.

Temporal Difference Learning

当 $\mathcal{S}$ 比较大或者连续时，我们构造损失函数

$$\min_{\phi} \mathcal{L}(\phi) = \mathbb{E}\left[\frac{1}{2}\left(V_\phi(s) - V^\pi(s)\right)^2\right] = \mathbb{E}\left[\left(V_\phi(s) - r_0 - \gamma V^\pi(s_1)\right)^2\right] \tag{76}$$

对应的梯度为

$$\nabla_\phi \mathcal{L}(\phi) = \mathbb{E}\left[\left(V_\phi(s) - r_0 - \gamma \boxed{V^\pi(s_1)}\right) \nabla_\phi V_\phi(s) \mid s_0 = s\right] \tag{77}$$

令

$$g := \left(V_\phi(s) - r_0 - \gamma \boxed{V^\pi(s_1)}\right) \nabla_\phi V_\phi(s) \tag{78}$$

但是，这里存在的问题在于，我们使用了 $V^\pi(s_1)$，而这个值是未知的，因此，一个做法是使用当前的 value function V_ϕ 来进行代替，即

$$\nabla_\phi \mathcal{L}(\phi) \approx \mathbb{E}\left[\left(V_\phi(s) - r_0 - \gamma V_\phi(s_1)\right) \nabla_\phi V_\phi(s) \mid s_0 = s\right] \tag{79}$$

Temporal Difference Learning–Naive version

这样，我们结合 TD 和 GSD 的算法就变成了

Algorithm 2: Value function approximation with TD (not correct)

while *not converged* **do**

$s_0 \sim p_0$, $a_0 \sim \pi(\cdot \mid s_0)$, $(r_0, s_1) \sim p(\cdot, \cdot \mid s_0, a_0)$;

$$y = \begin{cases} r_0 + \gamma V_\phi(s_1), & \text{if } s_1 \neq \text{<term>} \\ r_0, & \text{otherwise} \end{cases};$$

$g = (V_\phi(s_0) - y)\nabla_\phi V_\phi(s_0)$;

update ϕ using g with an optimizer;

Temporal Difference Learning–Analysis

接下来，我们需要关注如何计算 g, 直接通过 $g = (V_\phi(s) - r_0 - \gamma V_\phi(s_1))\,\nabla_\phi V_\phi(s)$ 来进行计算，这在数学上是没有问题的，但是从自动微分的角度不对，比如 Pytorch 对应的实现应该是

```
pred = V_phi(s0)
target = r + gamma * V_phi(s1)
td_error = pred - target
loss = 0.5 * td_error ** 2
loss.backward()
```

可以看到，我们实际上求的梯度是

$$\nabla_\phi \left[\frac{1}{2}\left(V_\phi(s) - (r + \gamma V_\phi(s'))\right)^2\right] = (V_\phi(s) - (r + \gamma V_\phi(s'))(\nabla_\phi V_\phi(s) - \nabla_\phi V_\phi(s')) \tag{80}$$

这显然与 g 不相等

Temporal Difference Learning–Analysis

为了解决这个问题，我们需要使用 stop-gradient 的技巧来避免 $V_\phi(s')$ 参与反向传播，这个时候，我们的目标函数就变成了

$$\frac{1}{2}\left(V_\phi(s) - (r + \gamma\ \mathrm{sg}[V_\phi(s')])\right)^2 \tag{81}$$

其中 $\mathrm{sg}[\cdot]$ 是 stop-gradient operator，满足

$$\mathrm{sg}[x] = \begin{cases} x & \text{forward pass} \\ 0 & \text{backward pass} \end{cases} \tag{82}$$

这样其梯度就是

$$\nabla_\phi \left[\frac{1}{2}\left(V_\phi(s) - (r + \gamma\ \mathrm{sg}[V_\phi(s')])\right)^2\right] = (V_\phi(s) - (r + \gamma V_\phi(s'))\nabla_\phi V_\phi(s) = g \tag{83}$$

对应的 python 代码为

```
pred = V_phi(s0)
target = r + gamma * V_phi(s1)
td_error = pred - target.detach()  # stop gradient operator
loss = 0.5 * td_error ** 2
loss.backward()
```

Temporal Difference Learning–Algorithm

最终，我们得到了一个简单的 TD 算法，其伪代码为

Algorithm 3: Value function approximation with TD

while *not converged* **do**

  $s_0 \sim p_0$, $a_0 \sim \pi(\cdot \mid s_0)$, $(r_0, s_1) \sim p(\cdot, \cdot \mid s_0, a_0)$;

  $$y = \begin{cases} r_0 + \gamma \, \mathrm{sg}[V_\phi(s_1)], & \text{if } s_1 \neq \text{<term>} \\ r_0, & \text{otherwise} \end{cases};$$

  $g = (V_\phi(s_0) - y)\nabla_\phi V_\phi(s_0)$;

  update ϕ using g with an optimizer;

Remark

上述这种做法其实是 semi-gradient methods，这类方法在参数更新时，只计算部分梯度。虽然这种方法牺牲了严格的梯度下降性质，但是其能够保证更好的表现。

k-step TD learning

我们可以进一步推广 TD 到多步的场景，注意到

$$V^\pi(s) = \mathbb{E}^\pi[r_0 + \gamma V^\pi(s_1) \mid s_0 = s] \tag{84}$$

$$= \mathbb{E}^\pi[r_0 + \gamma \mathbb{E}^\pi[r_1 + \gamma V^\pi(s_1) \mid s_1] \mid s_0 = s] \tag{85}$$

$$= \mathbb{E}^\pi[\mathbb{E}^\pi[r_0 + \gamma r_1 + \gamma^2 V^\pi(s_2) \mid s_1] \mid s_0 = s] \tag{86}$$

由重期望定理 (Law of Total Expectation, see Appendix Frame 57)

$$\mathbb{E}^\pi[\mathbb{E}^\pi[X \mid s_1] \mid s_0 = s] = \mathbb{E}^\pi[X \mid s_0 = s] \tag{87}$$

因此：

$$V^\pi(s) = \mathbb{E}^\pi[r_0 + \gamma r_1 + \gamma^2 V^\pi(s_2) \mid s_0 = s] \tag{88}$$

重复这个过程 k 次，得到 k 步展开：

$$V^\pi(s) = \mathbb{E}^\pi\left[\sum_{i=0}^{k-1} \gamma^i r_i + \gamma^k V^\pi(s_k) \mid s_0 = s\right] \tag{89}$$

目录

1 Introduction

2 Reinforcement Learning: Basics

3 Reinforcement Learning: Bellman Equation

4 Reinforcement Learning: Policy Evaluation

5 Reinforcement Learning: Value based methods

6 Reinforcement Learning: Policy based methods

7 Reinforcement Learning: Actor-Critic methods

目录

1 Introduction

2 Reinforcement Learning: Basics

3 Reinforcement Learning: Bellman Equation

4 Reinforcement Learning: Policy Evaluation

5 Reinforcement Learning: Value based methods

6 Reinforcement Learning: Policy based methods

7 Reinforcement Learning: Actor-Critic methods

目录

1 Introduction

2 Reinforcement Learning: Basics

3 Reinforcement Learning: Bellman Equation

4 Reinforcement Learning: Policy Evaluation

5 Reinforcement Learning: Value based methods

6 Reinforcement Learning: Policy based methods

7 Reinforcement Learning: Actor-Critic methods

目录

1 Introduction

2 Reinforcement Learning: Basics

3 Reinforcement Learning: Bellman Equation

4 Reinforcement Learning: Policy Evaluation

5 Reinforcement Learning: Value based methods

6 Reinforcement Learning: Policy based methods

7 Reinforcement Learning: Actor-Critic methods

目录

1 Introduction

2 Reinforcement Learning: Basics

3 Reinforcement Learning: Bellman Equation

4 Reinforcement Learning: Policy Evaluation

5 Reinforcement Learning: Value based methods

6 Reinforcement Learning: Policy based methods

7 Reinforcement Learning: Actor-Critic methods

目录

1 Introduction

2 Reinforcement Learning: Basics

3 Reinforcement Learning: Bellman Equation

4 Reinforcement Learning: Policy Evaluation

5 Reinforcement Learning: Value based methods

6 Reinforcement Learning: Policy based methods

7 Reinforcement Learning: Actor-Critic methods

目录

1 Introduction

2 Reinforcement Learning: Basics

3 Reinforcement Learning: Bellman Equation

4 Reinforcement Learning: Policy Evaluation

5 Reinforcement Learning: Value based methods

6 Reinforcement Learning: Policy based methods

7 Reinforcement Learning: Actor-Critic methods

References I

Law of Total Expectation

TODO

Fixed Point Theorem*

Fixed Point Theorem

给定函数 $f: \mathbb{R}^n \to \mathbb{R}^n$, 如果 f 是一个 contraction mapping, 则 f 具有如下性质

1. Existence: 存在 fixed point $x^* \in \mathbb{R}^n$ 满足 $f(x^*) = x^*$.
2. Uniqueness: fixed point x^* 唯一。
3. Algorithm: 对任意 $x^0 \in \mathbb{R}^n$, 使用迭代算法 $x_{k+1} = f(x_k)$ 产生的序列 $\{x_k\}_{k=0}^{\infty}$ 收敛到 fixed point $x*$, 且收敛速度为指数级。

Fixed Point Theorem–Cauchy Sequence

Cauchy Sequence

一个序列 $\{x_k\}_{k=0}^{\infty}$ 被称为柯西列 (Cauchy sequence) 当且仅当对任意 $\epsilon > 0$, 都存在 $N > 0$, 使得

$$\|x_m - x_n\| < \epsilon, \forall m, n > N \tag{90}$$

定理

一个序列 $\{x_k\}_{k=0}^{\infty}$ 是收敛列当且仅当它是柯西列。

Fixed Point Theorem–Proof

证明.

我们首先证明由 $x_k = f(x_{k=1})$ 产生的序列 $\{x_k\}_{k=1}^{\infty}$ 是收敛的。注意到 f 是一个 contraction mapping, 因此

$$\|x_{k+1} - x_k\| = \|f(x_k) - f(x_{k-1})\| \leq \gamma \|x_k - x_{k-1}\| \tag{91}$$

迭代下去，我们就得到

$$\|x_{k+1} - x_k\| \leq \gamma \|x_k - x_{k-1}\| \leq \cdots \leq \gamma^k \|x_1 - x_0\| \tag{92}$$

现在我们证明序列 $\{x_k\}_{k=1}^{\infty}$ 是一个柯西列：

$$\begin{aligned} \|x_m - x_n\| &= \|x_m - x_{m-1} + x_{m-1} - \cdots - x_{n+1} + x_{n+1} - x_n\| \\ &\leq \sum_{i=n}^{m-1} \|x_{i+1} - x_i\| \leq \sum_{i=n}^{m-1} \gamma^i \|x_1 - x_0\| \leq \frac{\gamma^n}{1-\gamma} \|x_1 - x_0\|. \end{aligned} \tag{93}$$

从而，序列 $\{x_k\}_{k=1}^{\infty}$ 是一个柯西列, 因此也是一个收敛列。 □

Fixed Point Theorem–Proof

证明.

接下来，我们证明 $x^* = \lim_{k\to\infty} x_k$ 是 $f(x)$ 的不动点，注意到

$$\|f(x_k) - x_k\| = \|x_{k+1} - x_k\| \le \gamma^k \|x_1 - x_0\| \to 0, k \to \infty \tag{94}$$

我们有 $\lim_{k\to\infty} f(x_k) = \lim_{k\to\infty} x_k$，由于 contraction mapping 一定是连续的，因此我们就可以得到 $f(x^*) = x^*$. 然后，我们证明不动点唯一。假设还存在一个另外一个不动点 $x' \neq x^*$ 满足 $f(x') = x'$, 那么

$$\|x' - x^*\| = \|f(x') - f(x')\| \le \gamma \|x' - x^*\| \tag{95}$$

由于 $\gamma \in (0, 1)$, 因此上述等式当且仅当 $\|x' - x^*\| = 0$, 这与前面假设矛盾，因而不动点是唯一的。 □

Fixed Point Theorem–Proof

证明.

最后，我们证明 $x_{k+1} = f(x_k)$ 这个算法的收敛速度为指数级，注意到

$$\|x^* - x_n\| = \lim_{m\to\infty} \|x_m - x_n\| \leq \frac{\gamma^n}{1-\gamma}\|x_1 - x_0\| \tag{96}$$

因为 $\gamma < 1$, 因此收敛速度为指数级。 □